

Technical Document #808: Software Engineering - Comprehensive Overview

Technical Document #808: Software Engineering - Comprehensive Overview

Microservices architecture structures applications as independently deployable services. Each service owns its data and business logic.

API design defines how software components communicate. RESTful APIs and GraphQL are popular API design approaches.

Code review examines code changes before merging. It improves code quality, catches bugs, and shares knowledge across the team.

Domain-driven design (DDD) models software around business domains. It improves communication between technical and business stakeholders.

Design patterns provide reusable solutions to common software design problems. Singleton, factory, and observer patterns are widely used.

Data-driven decision making has become essential for modern businesses. Organizations that leverage data effectively gain a significant competitive advantage.

Cloud computing has democratized access to powerful computing resources. Startups can now access the same infrastructure as large enterprises.

Computer vision applications are becoming ubiquitous, from facial recognition to autonomous driving. Deep learning has enabled breakthroughs in visual understanding.

Key Takeaways:

- Continuous integration (CI) automatically builds and tests code changes.
- Technical debt represents the cost of choosing easy solutions over better ones.
- Design patterns provide reusable solutions to common software design problems.